

Question 1 (2 points)

Any NFA having n states can be converted to a DFA having at most (select the smallest possible value)

- A. 2^n states
- B. 3^n states
- C. $2n$ states
- D. n^2 states

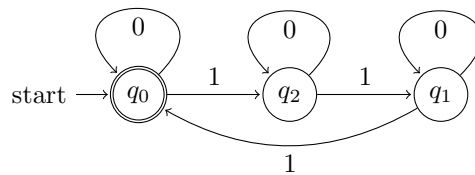
Option A is correct. Refer to the construction of a DFA for a given NFA (Theorem 2 of section 3.2 in Week 1 Lecture Notes). For a given NFA, the states of the constructed DFA correspond to the subsets of states of the NFA. If the NFA has n states, then there are 2^n subsets of states and hence 2^n states in the corresponding constructed DFA.

Question 2 (2 points)

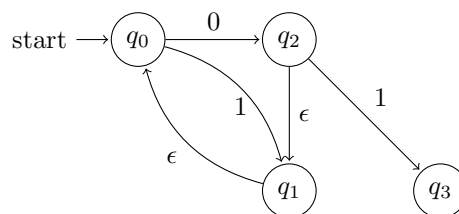
Consider the language $L = \{w \mid \text{the number of 1's in } w \text{ is divisible by 3}\}$ over the binary alphabet $\{0, 1\}$. What is the minimum number of states required for a DFA that accepts L ?

- A. 1
- B. 2
- C. 3
- D. 4

Option C is correct. We require 3 states to count the number of 1's modulo 3. The 3 states in the DFA correspond to the number of 1's in the input string being $(0 \bmod 3)$, $(1 \bmod 3)$ and $(2 \bmod 3)$. The minimal DFA that accepts L is shown below.

**Question 3** (2 points)

What is the ϵ -closure of state q_1 in the following NFA?

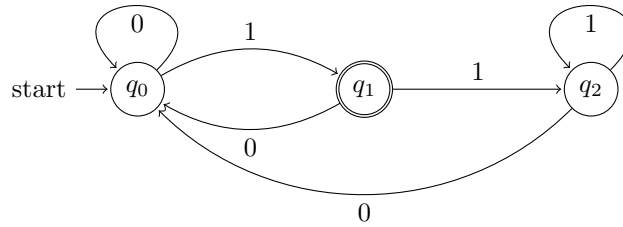


- A. $\{q_0, q_1, q_2\}$
- B. $\{q_0, q_1, q_2, q_3\}$
- C. $\{q_0, q_1\}$
- D. $\{q_1\}$

Option C is correct. ϵ -closure of a state q in an NFA refers to the set of all the states that are reachable from q via only ϵ -transitions.

Question 4 (2 points)

Which of the following strings is accepted by the DFA below?



- A. 01011101
- B. 00111100
- C. 10100011
- D. 11000010

Option A is correct. If we run the DFA on any of the other strings, we reach a state other than the accept state.

Question 5 (2 points)

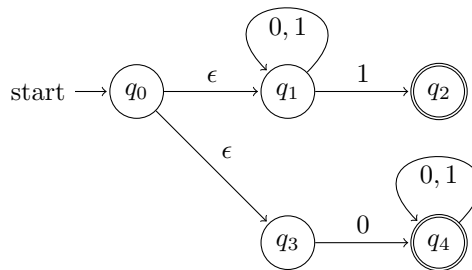
Let L_1 and L_2 be two regular languages. What can we say about the language $L = \{w_1w_2 \mid w_1 \in L_1, w_2 \in L_2\}$?

- A. L may or may not be regular
- B. L is regular
- C. L is not regular
- D. L is accepted by some NFA but there is no DFA that can accept L

Option B is correct. L is the concatenation of the languages L_1 and L_2 . Refer to the Closure Properties of Regular Languages (Section 3.3, Week 1 Lecture Notes)

Question 6 (2 points)

What is the language accepted by the following NFA?



- A. $\{w \in \{0,1\}^* \mid w \text{ ends with 0 and begins with 1}\}$
- B. $\{w \in \{0,1\}^* \mid w \text{ begins with 0 and ends with 1}\}$
- C. $\{w \in \{0,1\}^* \mid w \text{ ends with 0 or begins with 1}\}$
- D. $\{w \in \{0,1\}^* \mid w \text{ begins with 0 or ends with 1}\}$

Option D is correct. It is straightforward to verify that the given NFA has at least one computation path that leads to an accept state if and only if the input string begins with 0 or ends with 1. If the string ends with 1, then there is a computation path for the NFA that leads to state q_2 whereas if the string begins with 0, then there is a computation path for the NFA that leads to state q_4 . On the other hand, any computation path leading to state q_2 indicates that the input string ends with 1 and similarly, any computation path leading to state q_4 indicates that the input string begins with 0.

Question 7 (2 points)

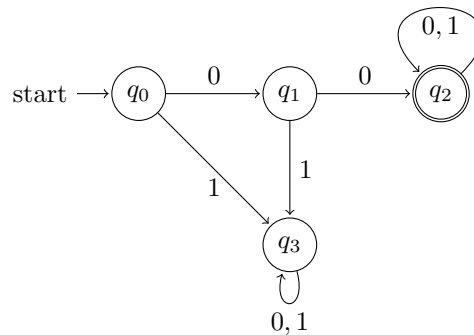
Consider the language

$$L = \{w \in \{0,1\}^* \mid w \text{ begins with } 00\}.$$

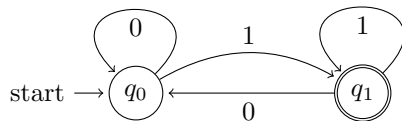
How many states will the minimal DFA for L have?

- A. 2
- B. 3
- C. 4**
- D. 5

Option C is correct. The minimal DFA for L is shown below.

**Question 8** (2 points)

Consider the following DFA.



What is the language accepted by this DFA?

- A. $\{w \in \{0,1\}^* \mid w \text{ has exactly one } 1\}$
- B. $\{w \in \{0,1\}^* \mid w \text{ ends with a } 1\}$**
- C. $\{w \in \{0,1\}^* \mid w \text{ begins with a } 1\}$
- D. $\{w \in \{0,1\}^* \mid w \text{ consists of only } 1\text{'s}\}$

Option B is correct. It is straightforward to verify that the given DFA leads to an accept state if and only if the input string ends with 1. If the string ends with 1, the DFA leads to state q_1 . On the other hand, if the DFA leads to state q_2 , it indicates that the input string ends with 1.

Question 1 (2 points)

Which of the following is equivalent to the set described by the regular expression 11^*00^* ?

- A. $\{1^m0^m \mid m > 0 \text{ is a natural number}\}$
- B. $\{1^m0^n \mid m, n > 0 \text{ are natural numbers}\}$
- C. $\{1^m0^m \mid m \geq 0 \text{ is a natural number}\}$
- D. $\{1^m0^n \mid m, n \geq 0 \text{ are natural numbers}\}$

Option B is correct. There can be any number of 1's and 0's in the strings belonging to this language. It is not necessary that the number of 0's and 1's be equal. But there should be at least one 0 and one 1 in the string.

Question 2 (2 points)

Let L be a regular language. Then what can we say about the language $L' = \{a_n a_{n-1} \dots a_1 \mid n \geq 0, a_1 a_2 \dots a_n \in L\}$?

- A. L' is regular
- B. L' is not regular
- C. L' may or may not be regular
- D. L' cannot be generated by a regular expression

Option A is correct. Refer to Closure Properties of Regular languages for Reversal (Section 5.1.2, Week 2 Lecture Notes)

Question 3 (2 points)

Any regular expression of size n can be converted to an NFA (with ϵ -transitions) having at most (select the smallest possible asymptotic upper bound)

- A. $O(n)$ states
- B. $O(n^2)$ states
- C. $O(2^n)$ states
- D. $O(1)$ states

Option A is correct. Refer to Converting an RE to NFA (Section 4.2.1, Week 2 Lecture Notes). For each symbol in the regular expression, we need to add at most 2 states to the corresponding NFA (given that we allow ϵ transitions). We also need to add at most 2 states to the corresponding NFA for each concatenation. There can be at most $n - 1$ concatenations in a regular expression with n symbols. Therefore, an NFA can be constructed corresponding to a regular expression of length (size) n with at most $4n - 2$ states, which is $O(n)$.

Question 4 (2 points)

Which of the following regular expressions generates the following language?

$$L = \{w \in \{0, 1\}^* \mid w \text{ begins with 1 and ends with 0}\}$$

Option A is correct. $(0 + 1)^*$ is the regular expression for the universal language. Therefore, $1(0 + 1)^*0$ is the regular expression for the language of strings beginning with 1 and ending with 0 (and having anything in between).

- A. $1(0 + 1)^*0$
- B. $(10)^*$
- C. 1^*0^*
- D. $1(01)^*0$

Question 5 (2 points)

Which of the following regular expressions is equivalent to the regular expression $(1 + \epsilon)(01)^*(0 + \epsilon)$?

- A. $(0 + \epsilon)(01)^*(1 + \epsilon)$
- B. $(1 + \epsilon)(10)^*(0 + \epsilon)$
- C. $(0 + \epsilon)(10)^*(1 + \epsilon)$
- D. $(01)^* + (10)^*$

Option C is correct. The key observations we make here are $1(01)^* = (10)^*1$ and $(01)^*0 = 0(10)^*$. Also we note that $(01)^* = 0(10)^*1 + \epsilon$ and $(10)^*10 + \epsilon = (10)^*$. We now use the commutativity of $+$ and distributivity of concatenation over $+$ in regular expressions and get the following derivation.

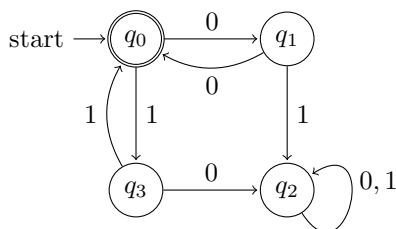
$$\begin{aligned} & (1 + \epsilon)(01)^*(0 + \epsilon) \\ &= 1(01)^*0 + 1(01)^* + (01)^*0 + (01)^* \\ &= (10)^*10 + (10)^*1 + 0(10)^* + 0(10)^*1 + \epsilon \\ &= ((10)^*10 + \epsilon) + (10)^*1 + 0(10)^* + 0(10)^*1 \\ &= (10)^* + (10)^*1 + 0(10)^* + 0(10)^*1 \\ &= (10)^*(1 + \epsilon) + 0(10)^*(1 + \epsilon) \\ &= ((10)^* + 0(10)^*)(1 + \epsilon) \\ &= (0 + \epsilon)(10)^*(1 + \epsilon) \end{aligned}$$

Question 6 (2 points)

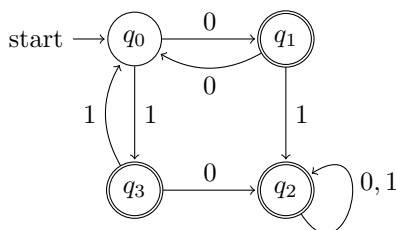
Which of the following regular expressions generates the complement of the language generated by the regular expression $(00 + 11)^*$?

- A. $(00 + 11)^*(0 + 1) + (00 + 11)^*(01 + 10)(0 + 1)^*$
- B. $(00 + 11)(0 + 1)^* + 0(00)^* + 1(11)^*$
- C. $(01 + 10)(0 + 1)^* + 0 + 1$
- D. $(00 + 11)(0 + 1)^* + (0 + 1)(01 + 10)^*$

Option A is correct. First construct the DFA for the given regular expression.



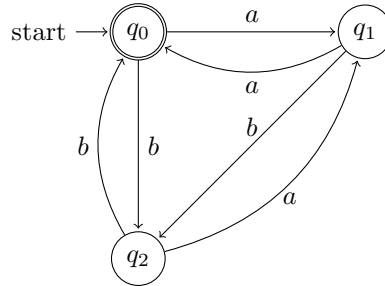
Then, construct the DFA for the complement language by making the accepting states in the DFA as non-accepting and the non-accepting states as accepting.



Now convert this DFA to regular expression following the process as described in Section 4.2.3 of Week 2 Lecture Notes.

Question 7 (2 points)

Which of the following regular expressions generates the same language as the given DFA?



- A. $(a(b + \epsilon)(ba)^*a + b(a + \epsilon)(ab)^*b)^*$
- B. $((b + \epsilon)a(ab)^*a + (a + \epsilon)b(ba)^*b)^*$
- C. $((b + \epsilon)a(ba)^*a + (a + \epsilon)b(ab)^*b)^*$
- D. $(a(a + \epsilon)(ba)^*a + b(b + \epsilon)(ab)^*b)^*$

Option C is correct. Convert the DFA to regular expression following the process as described in Section 4.2.3 of Week 2 Lecture Notes.

Question 8 (2 points)

Let R and S be two regular expressions. Which of the following is equivalent to the regular expression $S(S + RS)^*$?

- A. $R(SR + R)^*$
- B. $(RR^*S^*S^*)^*$
- C. $(S + SR)^*R$
- D. $RR^*S(RR^*S)^*$

None of the options is correct. This question will be reevaluated. That is, everyone will get marks for this question.

There is a typo in Option C, which was supposed to be the correct answer. Option C was intended to be $(S + SR)^*S$.

For any two regular expressions P and Q , $(PQ)^*$ can be written as $P(QP)^*Q + \epsilon$. Using this principle and the commutativity of $+$ and distributivity of concatenation over $+$ in regular expressions, we have the following derivation.

$$\begin{aligned}
 & S(S + RS)^* \\
 &= S((R + \epsilon)S)^* \\
 &= S((R + \epsilon)(S(R + \epsilon))^*S + \epsilon) \\
 &= S(R + \epsilon)(SR + S)^*S + S \\
 &= (SR + S)(SR + S)^*S + S \\
 &= ((SR + S)(SR + S)^* + \epsilon)S \\
 &= (SR + S)^*S
 \end{aligned}$$

Question 1 (2 points)

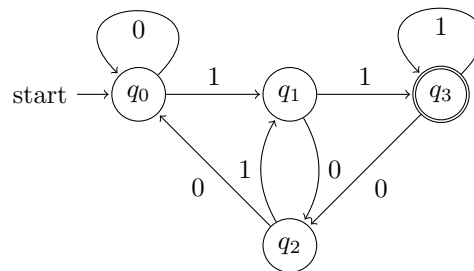
Let L be a regular language. Then according to the pumping lemma for regular languages, there exists an integer $p \geq 1$ such that every string $w \in L$ can be written as $w = xyz$ satisfying the following conditions:

- $|y| \geq 1$
- $|xy| \leq p$ and
 - A. $\forall n \geq 0, xy^n z \in L$
 - B. $\exists n \geq 0, xy^n z \in L$
 - C. $\forall n \geq 0, xyz^n \in L$
 - D. $\exists n \geq 0, x^n yz \in L$

Option A is correct. Refer to the Pumping Lemma (Section 6.1 in Week 3 Lecture Notes)

Question 2 (2 points)

Which states are equivalent in the following DFA?



- A. q_0 and q_1 only
- B. q_1 and q_2 only
- C. q_0 and q_2 only**
- D. q_0, q_1 and q_2

Option C is correct. Minimize the given DFA according to the process given in Section 7.1 in Week 3 Lecture Notes. You will find that the states q_0 and q_2 belong to the same equivalence class at the end of the process, but q_1 does not. That is, $q_0 \approx q_2$.

Question 3 (2 points)

Which of the following is a regular language?

- A. $\{0^m 1^m \mid m > 0 \text{ is a natural number}\}$
- B. $\{0^m 1^n \mid m, n > 0 \text{ are natural numbers}\}$**
- C. $\{0^m 1^m \mid m \geq 0 \text{ is a natural number}\}$
- D. $\{0^m 1^n \mid m, n \geq 0 \text{ are natural numbers such that } m \geq n\}$

Option B is correct. The regular expression corresponding to this language is 00^*11^* . It can be shown using the pumping lemma that the other languages are not regular.

Question 4 (2 points)

Which of the following is not a regular language?

- A. $\{0^m \mid m \text{ is a natural number}\}$
- B. $\{0^{2m} \mid m \text{ is a natural number}\}$
- C. $\{0^{3m+2} \mid m \text{ is a natural number } m \geq n\}$

D. $\{0^{m^2} \mid m \text{ is a natural number}\}$

Option D is correct. We can use pumping lemma to show that this language is not regular.

We will prove this via contradiction. Let's say this language is regular. Then it must satisfy the pumping lemma. Therefore there exists an integer $p \geq 1$ such that for some $k \leq p$, 0^{m^2+k} also belongs to the language. (The substring y in the pumping lemma has to be some string of 0's of length $k \leq p$ here and when we pump that substring y once into the string 0^{m^2} , we get 0^{m^2+k}). But for $m = p$, $m^2 + k \leq p^2 + p \leq (p+1)^2$. Thus, $m^2 + k$ is not the square of a natural number as it lies between the squares of two consecutive natural numbers p and $p+1$. This is a contradiction.

Question 5 (2 points)

Which of the following statements is not true?

- A. Any regular language satisfies the pumping lemma
- B. Any language that does not satisfy the pumping lemma cannot be regular
- C. Any language that satisfies the pumping lemma must be regular.**
- D. A language that is not regular may also satisfy the pumping lemma.

Option C is correct. Refer to the Pumping Lemma (Section 6.1 in Week 3 Lecture Notes)

Question 6 (2 points)

Which of the following languages is not regular?

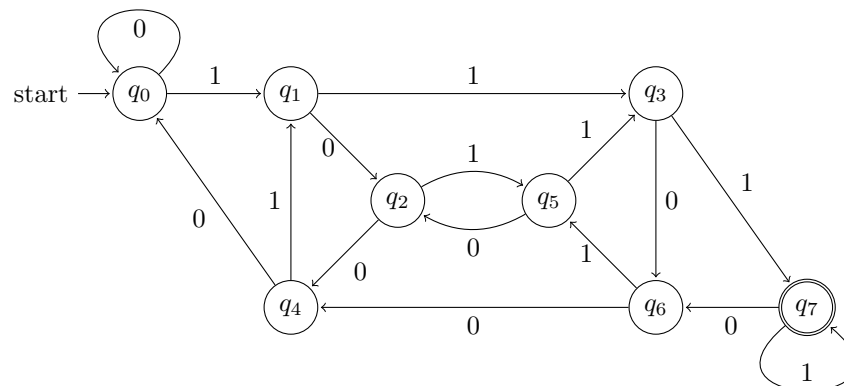
- A. $\{0^n \mid n \text{ is prime}\}$**
- B. $\{0^i 1^j \mid i, j \geq 0\}$
- C. Set of binary strings where every occurrence of 1 is followed by 0
- D. Set of binary strings such that the numbers represented by them in binary representation is divisible by 3

Option A is correct. We can use pumping lemma to show that this language is not regular.

We will prove this via contradiction. Let's say this language is regular. Then it must satisfy the pumping lemma. Therefore there exists an integer $p \geq 1$ such that for some $k \leq p$, 0^{n+kn} also belongs to the language. (The substring y in the pumping lemma has to be some string of 0's of length $k \leq p$ here and when we pump that substring y n times into the string 0^n , we get 0^{n+kn}). $n + kn = n(k+1)$ is not a prime number. This is a contradiction.

Question 7 (2 points)

How many states are there in the minimal DFA corresponding to the following DFA?



- A. 3

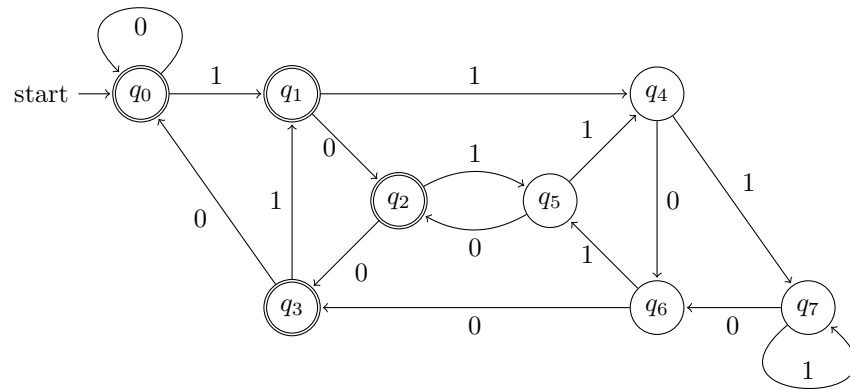
-
- B. 4
C. 6
D. 8

Option B is correct.

Minimize the given DFA according to the process given in Section 7.1 in Week 3 Lecture Notes. You will find that there are four equivalence classes of states for this DFA $\{q_0, q_2, q_4, q_6\}$, $\{q_1, q_5\}$, $\{q_3\}$ and $\{q_7\}$. Therefore, we can construct a minimal DFA with 4 states corresponding to these 4 equivalence classes.

Question 8 (2 points)

How many final/accepting states are there in the minimal DFA corresponding to the following DFA?



- A. 1
B. 2
C. 3
D. 4

Option C is correct.

Minimize the given DFA according to the process given in Section 7.1 in Week 3 Lecture Notes. You will find that there are six equivalence classes of states for this DFA $\{q_0, q_3\}$, $\{q_1\}$, $\{q_2\}$, $\{q_4, q_7\}$, $\{q_5\}$ and $\{q_6\}$. Out of these, three correspond to accept states- $\{q_0, q_3\}$, $\{q_1\}$ and $\{q_2\}$. Therefore, in the minimal DFA, there will be three accept states corresponding to these three equivalence classes.

Question 1 (2 points)

Consider the following context-free grammar:

$$S \rightarrow S0S1 \mid S1S0 \mid \epsilon$$

Which of the following is the language generated by this grammar?

- A. Set of all palindromes
- B. Set of all even numbers
- C. Set of all strings having either only 0's or only 1's
- D. Set of all strings having equal number of 0's and 1's**

Option D is the correct answer.

It is clear from the production rules of this grammar that any string generated by this grammar will have equal number of 0's and 1's.

Now we need to show that any string that has equal number of 0's and 1's can be generated by this grammar. We can prove this by induction. Let us assume that all strings of length at most $n - 1$ having equal number of 0's and 1's can be generated using the production rules of this grammar. Let us consider a string x of length n such that it has equal number of 0's and 1's. Let us assume that x ends with a 1 (the other case where x ends with 0 is symmetrical). Now consider the first 0 after 1 in x (counting from right to left). If there is nothing between this 0 and the ending 1, then the remaining string on the left of 0 also must have equal number of 0's and 1's and hence can be generated using the production rules as follows $S \rightarrow S0S1 \rightarrow S0\epsilon 1 \rightarrow S01$. The remaining string can be generated from S (by induction hypothesis). If this is not the case, we try to find the next 0 (moving from right to left) and see if the string between this 0 and the ending 1 has equal number of 0's and 1's. We keep on doing this until we find a 0 which divides x into two substrings p and q (i.e. $x = p0q1$) such that p and q both have equal number of 0's and 1's. We can then use the production rule $S \rightarrow S0S1$ to generate this string. The existence of such a 0 in x is guaranteed because as we move on from right to left, adding more 0's and 1's, the first time the number of 0's and 1's become equal has to be when we see a 0 (because we start with a 1).

Question 2 (2 points)

Consider the following grammar

$$\begin{aligned} S &\rightarrow PQ \mid \epsilon \\ P &\rightarrow PP \mid \epsilon \\ Q &\rightarrow QQ \mid \epsilon \end{aligned}$$

Which of the following regular expressions generates the same language as the given grammar?

- A. 0^*1^***
- B. $0^* + 1^*$
- C. $(0 + 1)^*$
- D. 00^*11^*

There is a typo in this question. It will be reevaluated in due time.

The second and third production rules were supposed to be $P \rightarrow PP \mid 0 \mid \epsilon$ and $Q \rightarrow QQ \mid 1 \mid \epsilon$. Option A i.e. 0^*1^* would be the correct answer then.

Question 3 (2 points)

Consider the following grammar

$$S \longrightarrow 0AB \mid 1BA \mid \epsilon$$

$$A \longrightarrow 0A0 \mid 1A1 \mid \epsilon$$

$$B \longrightarrow 0B1 \mid 1B0 \mid \epsilon$$

Which of the following strings does not belong to the language generated by this grammar?

A. 0011010

B. 0101001

C. 1101000

D. 1011001

Option B is the correct answer.

The variable A generates even-length strings that are palindromes and the variable B generates even-length strings that are anti-palindromes (i.e. none of the corresponding characters match in the string and its reverse). Therefore, S generates strings that are of one of the following forms:

- 1) Begins with 0, an even-length palindrome in the middle, an even-length anti-palindrome at the end.
- 2) Begins with 1, an even-length anti-palindrome in the middle, an even-length palindrome at the end.

Option A can be imagined as $0(0110)(10)$. 0110 is an even-length palindrome and 10 is an even-length anti-palindrome.

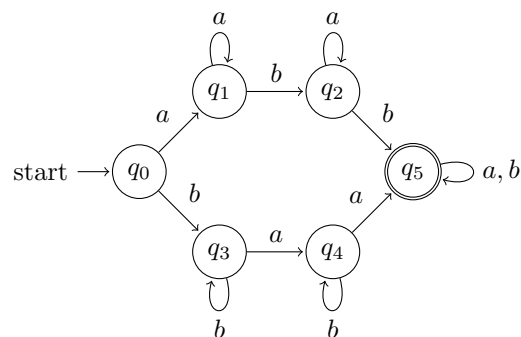
Option C can be imagined as $1(1010)(00)$. 1010 is an even-length anti-palindrome and 00 is an even-length palindrome.

Option D can be imagined as $1(01)(1001)$. 01 is an even-length anti-palindrome and 1001 is an even-length palindrome.

Option B cannot be imagined in any of these forms. This is because the substring 101001 is not an anti-palindrome and does not have a (non-empty) prefix that is a palindrome.

Question 4 (2 points)

Consider the following DFA.



Which of the following context-free grammars generate the same language as the given DFA?

-
- A. $S \rightarrow aA \mid bB$
 $A \rightarrow aA \mid bA'$
 $B \rightarrow bB \mid aB'$
 $A' \rightarrow aA' \mid bC$
 $B' \rightarrow bB' \mid aC$
 $C \rightarrow aC \mid bC \mid \epsilon$
- B. $S \rightarrow aA \mid bB$
 $A \rightarrow bA \mid aA'$
 $B \rightarrow aB \mid bB'$
 $A' \rightarrow aA' \mid bC$
 $B' \rightarrow bB' \mid aC$
 $C \rightarrow aC \mid bC \mid \epsilon$
- C. $S \rightarrow aA \mid bB$
 $A \rightarrow aA \mid bA'$
 $B \rightarrow bB \mid aB'$
 $A' \rightarrow aA' \mid bC$
 $B' \rightarrow bB' \mid aC$
 $C \rightarrow aC \mid bC \mid a \mid b$
- D. $S \rightarrow aA \mid bB$
 $A \rightarrow aA \mid bA'$
 $B \rightarrow bB \mid aB'$
 $A' \rightarrow bA' \mid aC$
 $B' \rightarrow aB' \mid bC$
 $C \rightarrow aC \mid bC \mid \epsilon$

Option A is the correct answer.

Using the method given in Section 8.1.4 of Week 4 Lecture notes, we can derive a context-free grammar for the given DFA.

Question 5 (2 points)

Choose the CFG that generates the same language generated by the regular expression $(bab)^*$.

- A. $S \rightarrow babS \mid \epsilon$
 B. $S \rightarrow bS \mid aS \mid \epsilon$
 C. $S \rightarrow babS \mid bab$
 D. $S \rightarrow baS \mid abS \mid \epsilon$

Option A is correct.

Question 6 (2 points)

Which of the following context-free grammars is ambiguous?

- A. $S \rightarrow 0S1 \mid \epsilon$
 B. $S \rightarrow 0S1 \mid 0 \mid 1 \mid \epsilon$
 C. $S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \epsilon$
 D. $S \rightarrow S0 \mid 0 \mid 1 \mid \epsilon$

Option D is correct. Let us consider the string 0. It can be derived in two ways using this grammar:

- 1) $S \rightarrow 0$
- 2) $S \rightarrow S0$
 $\rightarrow 0$ (using $S \rightarrow \epsilon$)

Therefore, this grammar is ambiguous.

Question 7 (2 points)

Let D be a DFA with n states that accepts the language L . Let G be the context-free grammar having the least number of non-terminals that generates L . The number of non-terminals in G is at most (select the smallest possible value)

- A. n
- B. $2n$
- C. n^2
- D. 2^n

Option A is the correct answer.

Refer to Section 8.1.4 of Week 4 Lecture notes. We can construct a grammar with n non-terminals for a DFA with n states. (where each non-terminal in the grammar corresponds to a state in the DFA)

Question 8 (2 points)

Which of the following is the context free grammar that generates the set of all palindromes over the alphabet $\{0, 1\}$?

- A. $S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1$
- B. $S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \epsilon$
- C. $S \rightarrow 0S0 \mid 1S1 \mid \epsilon$
- D. $S \rightarrow 0S1 \mid 1S0 \mid \epsilon$

Option B is the correct answer. Palindromes are strings which when reversed remains unchanged. Note that for such strings, the last and the first character must be same. Also if we remove the first and last character from the string, the remaining string is also a palindrome. Using this idea, we construct the grammar as given in option B.

Note:- The production rules $S \rightarrow 0$ and $S \rightarrow 1$ are used to generate odd-length palindromes (eg.- 101) where there is a single middle character. On the other hand, the production rule $S \rightarrow \epsilon$ is used to generate even-length palindromes which don't have a single middle character.

Question 1 (2 points)

Let L be a context-free language. Then according to the pumping lemma for context-free languages, there exists an integer $p \geq 1$ such that every string $w \in L$ of length at least p can be written as $w = uvwxy$ satisfying the following conditions:

- $|vx| \geq 1$
- $|vwx| \leq p$ and
 - A. $\forall n \geq 0, uv^nwx^ny \in L$
 - B. $\exists n \geq 0, uv^nwx^ny \in L$
 - C. $\forall n \geq 0, uv^nwx^n y \in L$
 - D. $\exists n \geq 0, uv^nwx^ny \in L$

There is a typo in this question. This question will be reevaluated.

Question 2 (2 points)

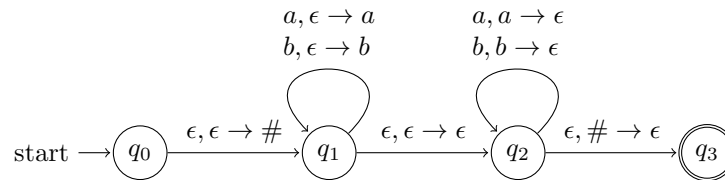
Which of the following languages is not context-free?

- A. $\{\}$
- B. $\{a^n b^n w \mid m, n \geq 0, w \in \{0, 1\}^*\}$
- C. $\{a^n b^n c^m d^m \mid m, n \geq 0\}$
- D. $\{ww \mid w \in \{0, 1\}^*\}$

Option D is correct. Refer to Week 5 Lecture Notes (Section 10.2 Example 2)

Question 3 (2 points)

Consider the following PDA.



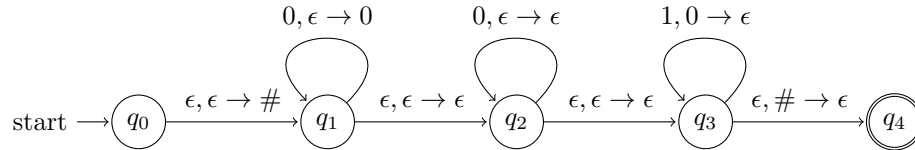
What is the language accepted by this PDA?

- A. Set of all strings of the form $a^n b^n$
- B. Set of all strings of the form $b^n a^n$
- C. **Set of all even-length palindromes over the alphabet $\{0, 1\}$**
- D. Set of all odd-length palindromes over the alphabet $\{0, 1\}$

Option C is correct. Compare the given PDA with the one in Example 2 in Section 11.1.5 of Week 5 Lecture Notes. The only difference is that the transitions $a, \epsilon \rightarrow \epsilon$ and $b, \epsilon \rightarrow \epsilon$ from state q_1 to q_2 are missing in the given PDA. This means the middle of the string can now only be ϵ and not a or b . In other words, there cannot be a single middle element in the accepted string. The accepted string must be divisible into exactly two halves of equal length such that the two halves are reverse of each other. Hence, this machine accepts only even length palindromes.

Question 4 (2 points)

Consider the following PDA.



Which of the following strings is not accepted by this PDA?

- A. 00000000
- B. 00000011
- C. 00001111
- D. 00111111**

Option D is correct. The given PDA accepts strings of the form $0^m 1^n$ such that $m \geq n$.

Question 5 (2 points)

Which of the following is context-free?

- A. $\{0^n \mid n \text{ is prime}\}$
- B. $\{ww^R \mid w \in \{0,1\}^* \text{ and } w^R \text{ is the reverse of } w\}$**
- C. $\{a^n c^m b^n d^m \mid m, n \geq 0\}$
- D. $\{a^n b^n c^n \mid n \geq 0\}$

Option B is correct. This is the set of all even length palindromes, which is known to be a context-free language. None of the other options are context-free languages. Refer to exercise 20 in section 10.2 of Week 5 Lecture Notes. We can show that these languages don't satisfy the pumping lemma for context-free languages.

Question 1 (2 points)

Consider the following languages

$$L_1 = \{a^n b^n a^m \mid n, m \geq 0\}$$

$$L_2 = \{a^n b^m a^n \mid n, m \geq 0\}$$

Which of the following languages is DCFL?

- A. $\overline{L_1}$
- B. $L_1 \cap L_2$
- C. $L_1 \cup L_2$
- D. $L_1 \cap \overline{L_2}$

Option A is correct. L_1 and L_2 are both DCFLs. DCFLs are closed under complementation, but not under intersection and union.

[$L_1 \cap L_2$ and $L_1 \cap \overline{L_2}$ are not context-free languages. We can show that these languages do not satisfy the pumping lemma for context-free languages. On the other hand, $L_1 \cup L_2$ is context-free but not deterministic context-free.]

Question 2 (2 points)

Which of the following languages is not necessarily context-free?

- A. Complement of a DCFL
- B. Intersection of a DCFL and a regular language
- C. Union of two DCFLs
- D. Intersection of two DCFLs

Option D is correct. DCFLs are closed under complementation and hence the complement of a DCFL is also DCFL and hence context-free. Intersection of a DCFL and a regular language is also a DCFL and hence context-free. Since context-free languages are closed under union, therefore the union of two DCFLs is also context-free. But CFLs as well as DCFLs are not closed under intersection. Therefore intersection of two DCFLs may not be context-free.

Question 3 (2 points)

Which of the following languages is necessarily deterministic context-free?

- A. Complement of a DCFL
- B. Intersection of a DCFL and a regular language
- C. Union of two DCFLs
- D. Intersection of two DCFLs

Both Option A and B are correct. This question will be reevaluated.

DCFLs are not closed under union and intersection but they are closed under complementation and intersection with a regular language.

Question 4 (2 points)

Let L_1 and L_2 be two languages such that $L_1 \subseteq L_2$. Consider the following statements.

- S_1 : If L_1 is DCFL then L_2 is also DCFL
- S_2 : If L_2 is DCFL then L_1 is also DCFL
- S_3 : Either both L_1 and L_2 are DCFL or neither L_1 nor L_2 are DCFL

Which of the following is correct?

-
- A. S_1 is true
 - B. S_2 is true
 - C. S_3 is true
 - D. None of S_1, S_2, S_3 is true**

Option D is correct. The subset of a DCFL is not necessarily a DCFL. Similarly the superset of a DCFL is also not necessarily a DCFL. Eg.- the non-context-free language $\{a^n b^n c^n\}$ is a subset of the DCFL $\{a^n b^n c^m\}$. Similarly the non-context-free language $\{a^n b^m c^p \mid m \neq n \neq p\}$ is a superset of the DCFL $\{a^n b^m \mid m \neq n \neq 0\}$ (considering $p = 0$).

Question 5 (2 points)

Let L_1 be a context-free language, L_2 be a DCFL. Which of the following is not true?

- A. $\overline{L_1} \cup L_2$ is necessarily context-free**
- B. $L_1 \cup \overline{L_2}$ is necessarily context-free
- C. $L_1 \cdot \overline{L_2}$ is necessarily context-free
- D. $L_1 \cup L_2$ is necessarily context-free

Option A is correct. Since DCFLs are closed under complementation, therefore $\overline{L_2}$ is DCFL (hence, context-free). Now since context-free languages are closed under union and concatenation, therefore $L_1 \cup L_2$, $L_1 \cdot \overline{L_2}$, $L_1 \cup \overline{L_2}$ are all context-free. On the other hand, context-free languages are not closed under complementation and hence $\overline{L_1}$ is not necessarily context-free and therefore $\overline{L_1} \cup L_2$ is also not necessarily context-free.

Question 6 (2 points)

Which of the following statements is not true about a deterministic Turing Machine?

- A. Equivalent to a nondeterministic Turing Machine in terms of computability
- B. Can go to multiple configurations from a given configuration**
- C. Can have multiple accept configurations
- D. Accepts a decidable language

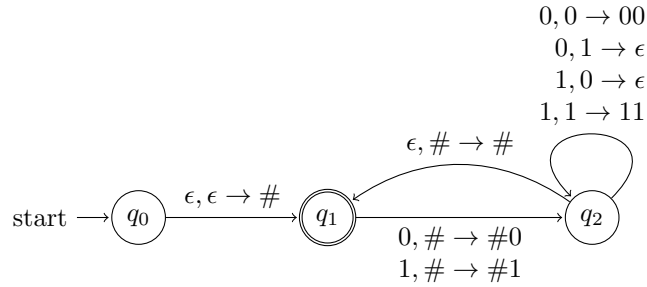
Option B is true. A deterministic Turing Machine can only move to a single configuration at once. (A non-deterministic Turing Machine can move to multiple configurations at once.)

Question 7 (2 points)

Which of the following is a deterministic context-free language?

- A. $\{ww^R \mid w \in \{0,1\}^* \text{ and } w^R \text{ is the reverse of } w\}$
- B. $\{ww \mid w \in \{0,1\}^*\}$
- C. $\{w \mid w \in \{0,1\}^* \text{ and } w \text{ has equal number of 0's and 1's}\}$**
- D. $\{a^n b^n c^m \text{ or } a^n b^m c^n \mid m, n \text{ are natural numbers}\}$

Option C is correct. Option A is the language of all even-length palindromes, which is known to be not a DCFL. (Refer to section 13.1 in Week 6 Lecture Notes). Option B is not even a context-free language. (Refer to section 10.2 in Week 5 Lecture Notes). Option D is not a DCFL although it is the union of two DCFLs (DCFLs are not closed under union.) Option C is a DCFL and here is a DPDA (by final state) accepting this language.



The above machine moves to the state q_1 whenever the stack gets empty. From state q_1 , it pushes a 0/1 (the next input symbol) and moves to state q_2 . When in state q_2 , the machine pushes the next input symbol onto the stack if it matches the symbol at the stack top. Otherwise, it pops one symbol out of the stack. The machine remains in state q_2 as long as the stack is not empty and once it gets empty, it moves to state q_1 . q_1 is the accept state, that is an input is accepted if and only if at the end of reading the input, the stack is empty. It is easy to see why this DPDA accepts all (and only those) strings that have equal number of 0's and 1's.

Question 8 (2 points)

Which of the following statements is not true about a Turing Machine?

- A. Can move across the work tape in both directions
- B. Can write on the work tape
- C. Has a fixed (independent of the input) number of states
- D. Has a fixed (independent of the input) number of configurations**

Option D is correct. The tape contents and how much of the tape is being used by the Turing machine depends on the input. Hence, the number of possible configurations of a Turing machine is not fixed. It depends on the input.

Question 1 (2 points)

The union of two decidable languages is

- A. context-free but not necessarily regular
- B. regular
- C. decidable but not necessarily context-free**
- D. recognizable but not necessarily decidable

Option C is correct. Decidable languages are closed under union.

Question 2 (2 points)

Which of the following languages is not Turing-recognizable?

- A. $\{\langle R \rangle \mid R \text{ is an RE and } L(R) = \emptyset\}$
- B. $\{\langle M \rangle \mid M \text{ is a NFA and } L(M) = \emptyset\}$
- C. $\{\langle G \rangle \mid G \text{ is a CFG and } L(G) = \emptyset\}$
- D. $\{\langle M \rangle \mid M \text{ is a TM and } L(M) = \emptyset\}$**

Option D is correct. Option A, B and C are Turing recognizable. (Refer to section 16.1 in Week 7 Lecture Notes). Option D is not Turing-recognizable (Refer to section 17.5 in Week 8 Lecture Notes)

Question 3 (2 points)

Consider the language

$$L = \{\langle G_1, G_2 \rangle \mid G_1, G_2 \text{ are CFGs and } L(G_1) = L(G_2)\}.$$

Then L is

- A. regular.
- B. decidable but not regular.
- C. Turing recognizable but not decidable.
- D. not Turing recognizable.**

Option D is correct. L is not decidable (Refer to section 16.2.3 in Week 7 Lecture Notes) and so is $\bar{L}$ (because decidable languages are closed under complementation). Now, $\bar{L}$ is Turing recognizable. This is because you can construct a Turing machine M that accepts and halts if $L(G_1) \neq L(G_2)$. M just needs to test for all possible strings, whether G_1 and G_2 both can generate the string. This problem is decidable (Refer to section 16.2.1 in Week 7 Lecture Notes). If $L(G_1) \neq L(G_2)$, then M is guaranteed to find some string which is generated by one of the grammars and not by the other. But M is not guaranteed to halt if $L(G_1) = L(G_2)$. By Proposition 26 (in section 17.2.1 of Week 8 Lecture Notes) or Theorem 23 (in section 15.1 of Week 7 Lecture Notes), L is not Turing recognizable.

Question 4 (2 points)

The intersection of two Turing-recognizable languages is

- A. context-free but not necessarily regular
- B. regular
- C. decidable but not necessarily context-free
- D. recognizable but not necessarily decidable**

Option D is correct. Turing recognizable languages are closed under intersection. (Refer to Section 15.1 of Week 7 Lecture Notes)

Question 5 (2 points)

What can we say about the following two languages?

$$\begin{aligned}A_{TM} &= \{\langle M, w \rangle \mid M \text{ is a TM that accepts the string } w\} \\E_{TM} &= \{\langle M \rangle \mid M \text{ is a TM and } L(M) = \emptyset\}\end{aligned}$$

- A. A_{TM} is Turing recognizable, E_{TM} is Turing recognizable
- B. A_{TM} is co-Turing recognizable, E_{TM} is Turing recognizable
- C. A_{TM} is Turing recognizable, E_{TM} is co-Turing recognizable**
- D. A_{TM} is co-Turing recognizable, E_{TM} is co-Turing recognizable

Option C is correct. Refer to chapter 17 (Week 8 Lecture Notes).

Question 6 (2 points)

Consider the following languages

$$\begin{aligned}L_1 &= \{\langle L \rangle \mid L \text{ is a regular language such that } |L| = 10\} \\L_2 &= \{\langle L \rangle \mid L \text{ is a regular language such that } |L| \text{ is infinity}\}\end{aligned}$$

Which of the following statement is true?

- A. L_1 and L_2 both are undecidable
- B. L_1 and L_2 both are decidable**
- C. L_1 is decidable but L_2 is undecidable
- D. L_1 is undecidable but L_2 is decidable

Option B is correct. Here, the assumption is that the description of a regular language $\langle L \rangle$ is given in the form of the DFA accepting the language. Considering the given DFA as a graph, we can compute all paths from the start state to the final states. If the number of such paths is 10, then we accept. Otherwise, we reject. Like this we can decide L_1 and hence L_1 is decidable.

Similarly, we can decide whether there is a loop across a path from the start state to some final state in the given DFA or not. If there is a loop then the cardinality of the language is infinity. Otherwise, it is not. Hence, L_2 is decidable.

Question 7 (2 points)

Which of the following statements is false?

- A. Complement of a regular language is also regular
- B. Complement of a DCFL is also a DCFL
- C. Complement of a decidable language is also decidable
- D. Complement of a Turing-recognizable language is also Turing recognizable**

Option D is correct. Refer to section 15.1 of Week 7 Lecture Notes.

Question 8 (2 points)

Consider the language

$$L = \{\langle D_1, D_2 \rangle \mid D_1, D_2 \text{ are DFAs and } L(D_1) = L(D_2)\}.$$

Then L is

- A. regular.
- B. decidable but not regular.**
- C. Turing recognizable but not decidable.
- D. not Turing recognizable.

Option B is correct. Refer to section 16.1.3 in Week 7 Lecture Notes.

Question 1 (2 points)

The class NP is defined as the class of problems that

- A. Can be solved in polynomial time by a nondeterministic Turing Machine
- B. Cannot be solved in polynomial time by a deterministic Turing Machine
- C. Can be solved in polynomial time by a deterministic Turing Machine
- D. Cannot be solved in polynomial time by a nondeterministic Turing Machine

Option A is correct. Refer to section 18.3.2 of Week 8 Lecture Notes.

Question 2 (2 points)

The class coNP is defined as

- A. $\{L \mid \bar{L} \in \text{NP}\}$
- B. $\{L \mid L \notin \text{NP}\}$
- C. $\{\bar{L} \mid L \subseteq L' \in \text{NP}\}$
- D. $\{\bar{L} \mid L \notin \text{NP}\}$

Option A is correct. Refer to section 18.3.4 of Week 8 Lecture Notes.

Question 3 (2 points)

$L \in \text{NP}$ if there exists constants $c > 0$ and $k \geq 0$, and a polynomial time Turing Machine V , such that

- A. $\exists x \in \Sigma^*, x \in L \iff \forall y \in \Sigma^* \text{ such that } |y| \leq c|x|^k \text{ and } V(x, y) = 1$
- B. $\forall x \in \Sigma^*, x \in L \iff \exists y \in \Sigma^* \text{ such that } |y| \leq c|x|^k \text{ and } V(x, y) = 1$
- C. $\forall x \in \Sigma^*, x \in L \iff \forall y \in \Sigma^* \text{ such that } |y| \leq c|x|^k \text{ and } V(x, y) = 1$
- D. $\exists x \in \Sigma^*, x \in L \iff \exists y \in \Sigma^* \text{ such that } |y| \leq c|x|^k \text{ and } V(x, y) = 1$

Option B is correct. Refer to section 18.3.2 of Week 8 Lecture Notes.

Question 4 (2 points)

$L \in \text{coNP}$ if there exists constants $c > 0$ and $k \geq 0$, and a polynomial time Turing Machine V , such that

- A. $\exists x \in \Sigma^*, x \in L \iff \forall y \in \Sigma^* \text{ such that } |y| \leq c|x|^k \text{ and } V(x, y) = 1$
- B. $\forall x \in \Sigma^*, x \in L \iff \exists y \in \Sigma^* \text{ such that } |y| \leq c|x|^k \text{ and } V(x, y) = 1$
- C. $\forall x \in \Sigma^*, x \in L \iff \forall y \in \Sigma^* \text{ such that } |y| \leq c|x|^k \text{ and } V(x, y) = 1$
- D. $\exists x \in \Sigma^*, x \in L \iff \exists y \in \Sigma^* \text{ such that } |y| \leq c|x|^k \text{ and } V(x, y) = 1$

Option C is correct. Consider the definition 18.3.1 given in section 18.3.2 of Week 8 Lecture Notes.

$L \in \text{NP}$ if there exists constants $c > 0$ and $k \geq 0$, and a polynomial time Turing Machine V , such that

$$\forall x \in \Sigma^*, x \in L \iff \exists y \in \Sigma^* \text{ such that } |y| \leq c|x|^k \text{ and } V(x, y) = 1$$

The above can be modified as follows

$L \in \text{coNP}$, that is, $\bar{L} \in \text{NP}$ if there exists constants $c > 0$ and $k \geq 0$, and a polynomial time Turing Machine V' , such that $\forall x \in \Sigma^*$,

$$x \in \bar{L} \iff \exists y \in \Sigma^* \text{ such that } |y| \leq c|x|^k \text{ and } V'(x, y) = 1$$

$$\text{i.e. } x \notin L \iff \exists y \in \Sigma^* \text{ such that } |y| \leq c|x|^k \text{ and } V'(x, y) = 1$$

$$\text{i.e. } \neg(x \notin L) \iff \neg(\exists y \in \Sigma^* \text{ such that } |y| \leq c|x|^k \text{ and } V'(x, y) = 1)$$

$$\text{i.e. } (x \in L) \iff \forall y \in \Sigma^* \text{ such that } |y| \leq c|x|^k \text{ and } V'(x, y) \neq 1$$

Now, if we consider V to be the Turing Machine that accepts the complement of the language accepted by V' , then we get the following statement.

$L \in \text{coNP}$ if there exists constants $c > 0$ and $k \geq 0$, and a polynomial time Turing Machine V , such that

$$\forall x \in \Sigma^*, (x \in L) \iff \forall y \in \Sigma^* \text{ such that } |y| \leq c|x|^k \text{ and } V(x, y) = 1$$

Question 5 (2 points)

Which of the following problems is not known to be in the complexity class P?

- A. Deciding whether a natural number is even
- B. Deciding whether a boolean formula is satisfiable**
- C. Deciding whether a graph is connected
- D. Deciding whether a binary string has even number of 1's

Option B is correct. This problem, also known as SAT, is an NP-complete problem. It is not known whether NP-complete problems can be solved deterministically in polynomial time or not.

Question 6 (2 points)

Which of the following problems is known to be in the complexity class coNP?

- A. Deciding whether a natural number is prime**
- B. Deciding whether a boolean formula is satisfiable
- C. Deciding whether a graph has a Hamiltonian path
- D. Deciding whether two graphs are isomorphic

Option A is correct. The complement of this problem, that is, deciding whether a natural number is composite is in NP. Refer to section 18.3.3 of Week 8 Lecture Notes.

Question 7 (2 points)

Let L be a language in the complexity class P. Which of the following is not known?

- A. $L \in \text{NP}$
- B. $L \in \text{coNP}$
- C. $L \in \text{NP} \cap \text{coNP}$
- D. $L \in \text{NP-complete}$**

Option D is correct. It is not known whether any problem in P (that is a problem that can be solved deterministically in polynomial time) is also in NP-complete. In fact, if that happens, then it will be proved that $\text{P} = \text{NP}$, which is a very famous open question.

Question 8 (2 points)

If $L \in \text{NTIME}(f(n))$, then

- A. $L \in \text{DTIME}(f(n))$.
- B. $L \in \text{DTIME}(\log f(n))$.
- C. $L \in \text{DTIME}(f(n)^2)$.
- D. $L \in \text{DTIME}(2^{O(f(n))})$.**

Option D is correct. Refer to section 20.1 of Week 8 Lecture Notes.

Question 9 (2 points)

The class NP is not known to be closed under which of the following operations?

- A. Union
- B. Intersection
- C. Concatenation
- D. Complementation**

Option D is correct. The complement of a language in NP belongs to the class coNP. If NP is closed under complementation, this would indicate $\text{NP} = \text{coNP}$, which is also a very famous open question.

Question 10 (2 points)

If $\text{coNP} \subseteq \text{NP}$ then which of the following is true?

- A. $\text{NP} = \text{coNP}$
- B. $\text{P} = \text{NP}$
- C. $\text{P} \neq \text{NP}$
- D. $\text{NP} \neq \text{coNP}$

Option A is correct.

Let us assume $\text{coNP} \subseteq \text{NP}$. Now, let $L \in \text{NP}$. Therefore,

$\bar{L} \in \text{coNP}$

$\implies \bar{L} \in \text{NP}$ (By our initial assumption)

$\implies L \in \text{coNP}$ (By definition of coNP)

This proves $\text{NP} \subseteq \text{coNP}$. Therefore, together with our initial assumption, this means $\text{NP} = \text{coNP}$.

Question 11 (2 points)

The class NP-hard is defined as the set of

- A. Problems which are reducible in polynomial time to some NP complete problem
- B. Problems which are reducible in polynomial space to some NP complete problem
- C. **Problems to which all problems in NP are reducible in polynomial time**
- D. Problems to which all problems in NP are reducible in polynomial space

Option C is correct. Refer to section 19.1 of Week 8 Lecture Notes.

Question 12 (2 points)

If a decision problem A is polynomial time reducible to another decision problem B , then which of the following statements is necessarily true?

- A. **If A is NP-hard then B is also NP-hard**
- B. If B is NP-hard then A is also NP-hard
- C. If A is in NP then B is in NP
- D. If A is NP-complete then B is also NP-complete

Option A is correct. Refer to section 19.2 of Week 8 Lecture Notes.